Project Topic: Design and Implementation of an Automated Short-Answer Grading System using Generative AI

Document Type: Technical Research & Architectural Justification

Date: January 2026

1. Introduction & Research Problem
Traditional Computer-Based Testing (CBT) systems often suffer from three critical failures: data loss during network interruptions, rigid navigation that increases student anxiety, and lack of immediate feedback. This project aims to solve these problems by architecting a Fault-Tolerant, AI-Powered Assessment Portal.

This report outlines the research-backed architectural decisions made to ensure the system is secure, scalable, and pedagogically effective.

2. Architectural Strategy: The Modular Monolith
Decision: The system follows a Modular Monolithic Architecture, separating the codebase into distinct logic layers (database, authentication, student_view, ai_engine).

Engineering Justification: According to recent software architecture standards, modular monoliths provide the simplicity of a single deployment unit while preventing the "spaghetti code" issues of traditional legacy systems. This approach enforces Separation of Concerns (SoC), ensuring that a UI update does not risk corrupting the grading logic.

Educational Benefit: In an educational setting, system downtime is unacceptable. Modular design allows for "Hot Fixes" (e.g., fixing a typo in a question) without needing to take down the entire authentication service.

3. Data Integrity: ACID Compliance for Examinations
Decision: We selected SQLite (Version 3.0) over simple file storage (CSV/JSON) to manage student records.

The "ACID" Requirement: Educational data requires strict adherence to Atomicity, Consistency, Isolation, and Durability (ACID).

Scenario: A student submits an exam at the exact moment a power failure occurs.

Solution: SQLite's atomic transaction support ensures that the submission is either fully saved or fully rolled back. It never leaves the database in a "half-written" corrupted state, which is a common failure point in non-relational storage.

Research Validation: Studies on e-learning reliability emphasize that "serverless" embedded databases like SQLite eliminate network latency errors during the critical data-save process, making them ideal for high-stakes testing environments.

4. Pedagogical User Experience (UX) Design
The student interface was not designed arbitrarily; it strictly follows Jakob Nielsen's 10 Usability Heuristics, the global standard for interface design.

A. Non-Linear Navigation (Heuristic #3: User Control)
Design: The "Question Map" allows students to skip questions and return later, rather than forcing a linear Question 1 -> Question 2 path.

Educational Research: Research by Hsu & Lin (2020) indicates that non-linear navigation significantly reduces test anxiety and cognitive load for students. It allows "strategic skipping," enabling students to answer confident questions first, which improves overall performance compared to rigid linear formats.

B. Visibility of System Status (Heuristic #1)
Design: Visual indicators (✓ for saved, 🔴 for current) provide real-time status updates.

Justification: In high-pressure exams, students often experience "Save Anxiety"—the fear that their work hasn't been recorded. Immediate visual feedback confirms the system's state, allowing the student to focus on recall rather than technical worry.

C. Error Prevention (Heuristic #5)
Design: The system replaces the "Instant Submit" button with a Review Phase.

Research: Accidental submissions are a leading cause of administrative overhead in e-exams. By implementing a "Gateway Page" that summarizes unanswered questions, we force a cognitive pause, ensuring students explicitly confirm their intent to finish.

5. Fault Tolerance: The "Draft" Persistence Layer
Decision: Implementation of a "Keystroke-Level" Auto-Save mechanism.

Technical Rationale: The system writes to a draft_answers table upon every interaction. This creates a Stateful Session even in a stateless web environment.

Real-World Scenario: If a student's browser crashes on Question 5, re-logging in fetches the data from the draft_answers table, restoring them exactly to where they left off. This resilience is critical for regions with unstable power or internet infrastructure.

6. Development Framework: Streamlit for Rapid Iteration
Decision: Usage of the Streamlit framework for the frontend interface.

Justification: Streamlit allows for the rapid transformation of Python-based data logic into interactive web applications. This is particularly suited for AI applications where the focus is on the backend logic (grading algorithms) rather than complex frontend CSS. It allows us to prototype the AI Grading Feedback Loop significantly faster than traditional web frameworks.

7. Conclusion

This pre-project research establishes that the proposed system is not merely a "coding project" but a scientifically engineered educational tool. By combining ACID-compliant data storage with research-backed pedagogical UX, the system mitigates the primary risks of online assessment (data loss and student anxiety) before the AI integration phase begins.